

EXECUTE TECH ACADEMY

DevOps & Cloud Engineering Programme

CAPSTONE PROJECT BRIEF

Deploying and Operating on AWS with Terraform

Duration: 14 July – 14 August 2026 (4 weeks)

Defence: Saturday 15 August 2026

Instructor: Olalekan Oladipupo

Source Code Guide: Bankole Adiamo

Overview

You will deploy the Execute Tech Academy website onto AWS ECS, delivered by a GitHub Actions pipeline, with every resource provisioned by Terraform. You will then provision an EKS cluster and deploy a small application to it that reads and writes a PostgreSQL database.

You are DevOps engineers, not application developers. You will not write application code. Every line of application code you need is either handed to you by Bankole or printed in this brief. Your work is containerisation, infrastructure, delivery, and operations.

NON-NEGOTIABLE: Every AWS resource must be created by Terraform. Anything you click into existence in the AWS Console does not count, will not be graded, and must be destroyed.

Know Your Application Before You Deploy It

Read this section carefully. Half the failures on this project come from not understanding what you are shipping.

The Execute Tech Academy website is a Create React App (CRA) single-page application. React 19, react-scripts 5. When you run `npm run build`, it compiles to a folder of static files — HTML, CSS, and JavaScript. That is all it is. There is no server. There is no runtime. Nothing in this application can open a database connection, and nothing in it ever will.

What it is	What that means for you
A static bundle	Once built, it is just files. It is served by a static file server (<code>npx serve -s build</code>).
No server-side runtime	It cannot talk to RDS. It cannot hold a secret. It cannot run code on your infrastructure.
Calls a backend elsewhere	<code>src/utils/api.js</code> points at <code>process.env.REACT_APP_API_URL</code> . That backend is NOT part of this project.
Serves 200 on every path	A static server returns the index page for any URL. Your ALB health check can only prove the container is up — not that the app works.

THE TRAP THAT WILL CATCH YOU: `REACT_APP_API_URL` is a BUILD-TIME variable. Create React App substitutes it into the JavaScript bundle when you run `npm run build`. It is NOT read at runtime. Setting it as an ECS task definition environment variable does absolutely nothing — the value was already frozen into the bundle when the image was built. If you need to change it, you rebuild the image. There is no other way.

AND THE SECURITY CONSEQUENCE: Because the value is baked into the bundle, ANY value you put in a `REACT_APP_*` variable is downloaded by every visitor and readable in their browser. Open DevTools, search the bundle, there it is. A database password in a `REACT_APP_*` variable is a password you have published on the internet. Never put a secret in one. You will be asked about this at defence.

Project 1 — The Website on ECS (60%)

1.1 Source Code

- Get the repository from Bankole Adiamo. He is the contact for anything about the application code. He is not the contact for your infrastructure.
- Fork it into your own GitHub repository. Do not push to his.
- Do not change application logic. Your job is delivery.
- Note the structure: the app lives in the frontend/ subdirectory, not at the repo root. Your Dockerfile and your pipeline paths must account for that.

1.2 Stage 1 — Build and Run It Locally (hard gate)

You do not touch AWS until it runs in Docker on your own machine.

```
cd frontend
npm install
npm run dev          # react-scripts dev server, localhost:3000
npm run build        # produces the static build/ folder – LOOK INSIDE IT
npx serve -s build -l 3000
```

Then write the Dockerfile yourself. Multi-stage. Stage one builds the bundle with Node; stage two serves those static files with nginx. The final image must contain NO node_modules and NO source code — only the built assets and nginx.

```
# Stage 1 – build
FROM node:20-alpine AS build
WORKDIR /app
COPY frontend/package*.json ./
RUN npm ci
COPY frontend/ .
ARG REACT_APP_API_URL
ENV REACT_APP_API_URL=$REACT_APP_API_URL
RUN npm run build

# Stage 2 – serve
FROM nginx:alpine
COPY --from=build /app/build /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

Notice the ARG. The API URL is passed in at BUILD time (docker build --build-arg), because that is the only time CRA can see it. If you find yourself trying to pass it at docker run time, stop and re-read the trap above.

nginx.conf — a single-page app needs every route to fall back to index.html, or a browser refresh on /courses returns 404:

```
server {
    listen 80;
    root /usr/share/nginx/html;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

Build it and prove it works, including a hard refresh on a deep route like /courses:

```
docker build --build-arg REACT_APP_API_URL=http://localhost:8080 -t eta-web .
docker run -p 8080:80 eta-web
```

WHY THIS GATE: If it will not run in Docker on your laptop, you will lose three days debugging ECS task failures that have nothing to do with ECS. Fix it where the feedback loop is two seconds, not five minutes.

1.3 Stage 2 — Terraform the Infrastructure

Resource	Requirement
VPC	Custom VPC. Public + private subnets across 2 Availability Zones.
ECR	One repository for the website image. A second for the Project 2 app.
ECS	Fargate cluster, task definition, service. Tasks run in PRIVATE subnets. Container port 80.
ALB	Public subnets. Target group to the ECS service. Health check path: /
RDS	PostgreSQL 16, db.t3.micro, private subnets, NOT publicly accessible. See the note below — read it.
Secrets Manager	The RDS master credentials. Generated by Terraform, stored here, never written to a .tf file.
IAM	Task execution role and task role. Least privilege.
Security Groups	ALB: 80/443 from internet. ECS: only from ALB. RDS: only from ECS, and later from the EKS nodes.

BE HONEST ABOUT THE RDS IN PROJECT 1: The website is static and cannot use this database. You are provisioning it here because Project 2 will use it, and because provisioning a private, credential-managed database correctly is a skill you are being assessed on. You must prove it is reachable — exec into your running ECS task and connect to it with psql. If you cannot, your security groups or subnets are wrong, and you need to know that NOW, not in week 4 when the EKS pods will not start.

Prove the database is alive from inside the ECS task:

```
aws ecs execute-command --cluster <cluster> --task <task-id> \  
  --container web --interactive --command "/bin/sh"  
  
# then, inside the container:  
apk add --no-cache postgresql-client  
psql -h <rds-endpoint> -U <user> -d <db> -c 'SELECT version();'
```

(ECS Exec must be enabled on the service in your Terraform. That is part of the task.)

HARD FAIL: Any password, secret, or connection string hardcoded in a .tf file, a Dockerfile, a workflow file, or committed to Git is an automatic fail. Terraform state contains secrets too — use a remote S3 backend with encryption enabled, and never commit state files.

1.4 Stage 3 — GitHub Actions Pipeline

Runs on push to main:

- Checkout. Install dependencies (remember: working directory is frontend/).
- Run npm run lint. The pipeline fails if linting fails.
- Build the Docker image, passing REACT_APP_API_URL as a --build-arg. Tag with the git commit SHA, never 'latest'.
- Authenticate to AWS using OIDC. Long-lived AWS access keys stored as GitHub secrets are not acceptable.
- Push to ECR. Register a new task definition revision. Update the ECS service. Wait for the deployment to stabilise.

WHY OIDC: Static AWS keys sitting in a CI system are how organisations get breached. You will do this the way a bank does it, because some of you will end up working at one.

1.5 Acceptance Criteria — Project 1

- The website loads in a browser at the ALB DNS name. Not an EC2 IP. Not localhost.

- A hard refresh on a deep route (for example /courses) loads correctly and does not 404. This proves your nginx fallback is right.
- You can exec into the running ECS task and reach RDS with psql.
- A push to main triggers the pipeline and the change is live at the ALB URL with no manual step from you.
- terraform destroy followed by terraform apply rebuilds the entire environment and it works again.
- No secret anywhere in the repository, and no secret in the JavaScript bundle. You will be asked to open DevTools and search it.

Project 2 — EKS with Terraform (40%)

2.1 The Application — Given To You

You are not writing an application. Here it is. Two endpoints. It writes a visit to PostgreSQL and reads the count back. Its only job is to prove that a pod in your cluster can reach the database you built in Project 1, using a Kubernetes Secret.

app.js

```
const express = require('express');
const { Pool } = require('pg');

const app = express();
const pool = new Pool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  port: 5432,
});

pool.query('CREATE TABLE IF NOT EXISTS visits (id SERIAL PRIMARY KEY, seen_at TIMESTAMP
DEFAULT NOW())');

// the readiness probe hits this – it checks the DB, not just the process
app.get('/healthz', async (req, res) => {
  try {
    await pool.query('SELECT 1');
    res.status(200).send('ok');
  } catch (err) {
    res.status(500).send('db unreachable');
  }
});

app.get('/', async (req, res) => {
  await pool.query('INSERT INTO visits DEFAULT VALUES');
  const r = await pool.query('SELECT COUNT(*) FROM visits');
  res.send(`Hello from EKS. Visits: ${r.rows[0].count}`);
});

app.listen(3000, () => console.log('listening on 3000'));
```

package.json

```
{
  "name": "eks-visit-counter",
  "version": "1.0.0",
```

```
"scripts": { "start": "node app.js" },
"dependencies": { "express": "^4.19.2", "pg": "^8.11.5" }
}
```

Dockerfile

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --omit=dev
COPY app.js .
EXPOSE 3000
CMD ["node", "app.js"]
```

NOTE THE DIFFERENCE: Unlike the React site, THIS app reads its configuration at RUNTIME. `process.env.DB_HOST` is read when the container starts, so a Kubernetes Secret works exactly as you expect. Understanding why one app can take runtime config and the other cannot is the central lesson of this capstone.

2.2 Provision the Cluster

- Terraform an EKS cluster: VPC (reuse Project 1's or build a new one), cluster, managed node group, IAM roles.
- Use the `terraform-aws-modules/eks/aws` module. You are not expected to write EKS from raw resources — you ARE expected to explain what the module created. You will be asked.
- Node group: 2 nodes, `t3.small`. Nothing larger.
- Allow the EKS node security group into the RDS security group on port 5432. This is the join between your two projects. Get it right.
- Prove access: `kubectl get nodes` returns Ready nodes.

2.3 Deploy It

Build the image, push it to ECR, deploy with manifests containing:

- A Deployment, 2 replicas, with resource requests and limits set.
- A Secret holding `DB_HOST`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, injected as environment variables. The values come from what Terraform put in Secrets Manager.
- A readiness probe on `/healthz`. A pod that cannot reach the database must not receive traffic.
- A Service of type `LoadBalancer` so it is reachable from a browser.

2.4 Acceptance Criteria — Project 2

- `kubectl get nodes` — all Ready.
- `kubectl get pods` — Running. No `CrashLoopBackOff`. No `OOMKilled`.
- Loading the Service's external address in a browser shows the visit count.
- Refresh the page. The number goes up. That is your proof the pod is genuinely writing to the RDS instance you built in week 2.
- `kubectl describe pod` shows your resource limits and your readiness probe.
- Deliberately break the Secret (wrong password). Show that the readiness probe fails, the pod goes `NotReady`, and the Service stops sending it traffic. Then fix it. You will do this live at defence.

Cost Discipline — Read This Twice

THIS COSTS REAL MONEY: EKS bills roughly \$0.10 per hour for the control plane before a single node runs. Add nodes, RDS, a NAT Gateway and an ALB, and an environment left running over a weekend costs more than you expect. This is your own money.

- Set a billing alarm on DAY ONE, before you provision anything.
- Run terraform destroy at the end of every working session. Not at the end of the project — every session.
- Capture your evidence — screenshots, terminal output — BEFORE you destroy.
- Smallest sizes only. db.t3.micro for RDS. t3.small for EKS nodes.
- The NAT Gateway is billed hourly and is one of the largest line items here. Know that it is there.
- Proof of terraform destroy is a required deliverable. Resources still running at defence is a mark against you.

Timeline — 14 July to 14 August 2026

Week	Dates	Milestone
Week 1	14 – 20 July	Repo forked. Billing alarm set. App builds and runs locally. Multi-stage Dockerfile written, nginx fallback working, deep-route refresh does not 404. Remote S3 state backend configured.
Week 2	21 – 27 July	Terraform: VPC, subnets, security groups, RDS, Secrets Manager. Applied cleanly. RDS credentials generated by Terraform into Secrets Manager.
Week 3	28 July – 3 Aug	Terraform: ECR, ECS Fargate (with Exec enabled), ALB. Site live at the ALB URL. psql to RDS from inside the task succeeds. GitHub Actions pipeline with OIDC deploying on push. PROJECT 1 COMPLETE.
Week 4	4 – 10 Aug	EKS cluster via Terraform. Counter app containerised, pushed to ECR, deployed with Secret, probe and limits. Counter increments in the browser against the Project 1 RDS.
Buffer	11 – 14 Aug	README and runbook written. Screenshots captured. Everything destroyed. Submit by 23:59 on 14 August.
Defence	Sat 15 Aug	Live demonstration.

THE BUFFER IS NOT FREE TIME: It is there because something WILL break in week 3 or 4 and eat two days. If you reach 11 August with nothing left to fix, you are the exception, not the rule. Do not plan to start week 4's work in the buffer.

Troubleshooting — Symptom to Cause

You will not write or debug application code on this project. Every file you need is printed in this brief or handed to you by Bankole. You do not need to open src/ at any point.

But you WILL hit failures, and most of them look like application bugs when they are not. This table is the difference between a wasted evening and a five-minute fix. When something breaks, come here first.

Local Docker

Symptom	Cause	Fix
docker build fails at npm ci	Dockerfile COPY path is wrong. The app is in frontend/, not the repo root.	COPY frontend/package*.json ./ — not COPY package*.json ./
Container runs but browser shows nothing	You mapped the wrong port. nginx listens on 80, not 3000.	docker run -p 8080:80, not -p 8080:3000
Homepage loads, but refreshing /courses gives 404	nginx.conf is missing or not copied into the image.	Confirm the try_files line and that the Dockerfile COPYs nginx.conf into /etc/nginx/conf.d/default.conf
Site loads but API calls fail	REACT_APP_API_URL was not passed as a build argument, so the bundle has 'undefined' baked in.	docker build --build-arg REACT_APP_API_URL=... Rebuild the image. You cannot fix this at runtime.

ECS

Symptom	Cause	Fix
Task starts then immediately stops	Look at the STOPPED task's reason in the console or CLI. Usually the image failed to pull, or the container exited.	<code>aws ecs describe-tasks</code> — read <code>stoppedReason</code> . Check CloudWatch Logs for the container output.
Task stuck in PENDING	Fargate cannot pull the image. Private subnet with no NAT Gateway, or no VPC endpoint for ECR.	Confirm your private subnets route 0.0.0.0/0 to the NAT Gateway.
ALB target unhealthy, task is running	Port mismatch. The target group is pointing at a port nothing is listening on.	Target group port must be 80, matching the nginx container port.
ALB returns 503	No healthy targets. The task is not running, or every target failed its health check.	Check the ECS service events and the target group health tab.
Cannot reach RDS from the task	Security group. The RDS SG must allow 5432 FROM the ECS task security group — not from a CIDR block.	Reference the ECS SG id as the source in the RDS SG ingress rule.
<code>aws ecs execute-command</code> fails	ECS Exec is not enabled on the service, or the task role lacks SSM permissions.	<code>enable_execute_command = true</code> on the service, plus the SSM policy on the task role.

GitHub Actions

Symptom	Cause	Fix
Could not assume role / no credentials	OIDC trust policy is wrong, or the workflow is missing permissions: <code>id-token: write</code> .	Add the permissions block. Check the trust policy subject matches your repo and branch exactly.
<code>npm ci</code> fails in the pipeline	Working directory. The workflow is running at the repo root, not in <code>frontend/</code> .	Set <code>working-directory</code> : <code>frontend</code> on the step, or <code>cd</code> into it.
Pipeline is green but the site did not change	You pushed a new image but did not register a new task definition revision, or did not update the service.	The pipeline must register a new revision AND update the service. Tag by commit SHA, never 'latest'.

EKS

Symptom	Cause	Fix
<code>kubectl</code> : connection refused / unauthorized	kubeconfig not updated for the cluster.	<code>aws eks update-kubeconfig --name <cluster> --region <region></code>
Pod stuck in Pending	No node has capacity, or the node group did not come up.	<code>kubectl describe pod</code> — read the Events. <code>kubectl get nodes</code> .
Pod is Running but never Ready	The readiness probe on <code>/healthz</code> is failing. The pod cannot reach the database.	This is almost always the Secret (wrong password/host) or the RDS security group not allowing the EKS node SG on 5432.
CrashLoopBackOff	The container is exiting. Read the logs.	<code>kubectl logs <pod> --previous</code> — the crash reason is in the previous container's output.
OOMKilled	The memory limit is too low for the app.	<code>kubectl describe pod</code> , look for Last State: Terminated, Reason: OOMKilled. Raise the memory limit.
LoadBalancer Service has no external IP	The AWS load balancer controller is still provisioning, or the subnets are not tagged for it.	Wait 2-3 minutes. Then check subnet tags: <code>kubernetes.io/role/elb = 1</code> on public subnets.

THE RULE: Read the error before you change anything. `kubectl describe pod`, `kubectl logs`, `aws ecs describe-tasks`, and the CloudWatch log group will tell you what is wrong. Guessing and re-applying Terraform until something works is not troubleshooting, and it will be obvious at defence that you did it.

Required Deliverables

One GitHub repository link containing:

Deliverable	Must Contain
Terraform code	All .tf files, organised into modules. No secrets. .gitignore excludes state files.
Dockerfile + nginx.conf	Multi-stage build for the React site. The SPA fallback config.
GitHub Actions workflow	The .github/workflows/ pipeline definition.
Kubernetes manifests	Deployment, Service, Secret, readiness probe, resource limits.
README.md	Architecture diagram. How to deploy from scratch. How to destroy. Written for someone who has never seen your project.
Runbook	How to check health. How to read logs. What to do when: the ECS task will not start, the ALB target is unhealthy, a deep route 404s, the pod is CrashLoopBackOff, the pod cannot reach RDS. Write it from bugs you actually hit.
Screenshots	Site at the ALB URL. A deep route loading after refresh. Green pipeline run. psql output from inside the ECS task. <code>kubectl get nodes</code> and <code>get pods</code> . Counter incrementing. terraform destroy output.

Grading Rubric

Criterion	Weight	Full Marks
Infrastructure as Code	25%	All Terraform. destroy + apply rebuilds cleanly. Modular, not one 800-line main.tf.
Security	20%	No secrets in Git. No secrets in the JS bundle. Secrets Manager used properly. OIDC not static keys. RDS private. Security groups reference each other, never 0.0.0.0/0.
CI/CD	20%	Runs on push, fails on lint errors, deploys automatically, tags by commit SHA, passes the build-arg correctly.
Kubernetes	20%	Cluster by Terraform. App running with probe, limits, Secret. Counter increments against the shared RDS.
Documentation	10%	README and runbook good enough for a stranger to deploy and troubleshoot your project.
Cost discipline	5%	Billing alarm set. Environment destroyed. Proof provided.

Defence — Saturday 15 August

You will demonstrate live. Expect to be asked to:

- Run `terraform destroy`, then `terraform apply`, and bring the whole thing back up while we watch.
- Explain why `REACT_APP_API_URL` had to be a build argument and could not be an ECS environment variable.
- Open DevTools on your deployed site, search the JavaScript bundle, and explain what a visitor can and cannot see.
- Explain every security group rule you wrote, and why.
- Show where your secrets live and prove they are not in Git.

- Explain what the EKS module created on your behalf.
- Explain how a pod in EKS reaches the database that was provisioned in Project 1.
- Break the Kubernetes Secret on purpose, show the readiness probe failing, and fix it.

FINAL NOTE: Copied Terraform falls apart the moment you are asked to explain a single line of it. Build it yourself, break it yourself, fix it yourself. That is the whole point of this exercise.